
Authorship Confounds in AI-Control Monitor Evaluation, and a Same-Generator Corpus

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Trusted monitors are tuned and ranked on stored pairs of code, pairing for one pro-
2 gramming problem an honest solution with a solution carrying a planted backdoor.
3 The two classes are built differently. Honest solutions are human-written, taken
4 from an existing dataset; every backdoor is written by a language model. The label
5 is therefore recoverable from authorship alone, and filtering to working solutions
6 and stripping comments does not remove it. On such a split, a trained bag-of-words
7 classifier scores above roughly four-fifths of the open-weight monitors ranked on it,
8 and a classifier reading no code at all, only problem metadata, scores above most
9 of them. The signal transfers across backdoor generators and is nearly as strong
10 on inert backdoors as on working ones, so no property of a backdoor explains it.
11 Regenerating only the honest half raises separability further. One model writing
12 both classes removes between a third and nine-tenths of it, on two generators that
13 differ because only one adopts the attack prompt’s layout; matching that prompt
14 removes most of the rest. An LLM monitor is inflated the same way, scoring 0.944
15 on stored pairs and 0.818 on firing backdoors from the same author. Stored-pair
16 evaluations should report partial-input and bag-of-words floors against permuta-
17 tion nulls, generate both classes from one model and one prompt, and drop inert
18 backdoors.

1 Introduction

20 The backbone protocol of AI control is trusted monitoring, where a weaker trusted model reads an
21 untrusted model’s output and flags sabotage [4]. Monitors are developed, tuned and ranked almost
22 entirely on *stored* honest/attack pairs, fixed corpora in which each programming problem contributes
23 an honest solution and a solution carrying a planted backdoor [12, 19, 9]. The two classes come
24 from different authors. Honest solutions are human-written, taken from APPS [7]; every backdoor is
25 written by a language model, and the corpus records which one. Separation credited to a monitor
26 may be separation between two authors.

27 That these pairs leak is known, and the corpora are preprocessed against it. The filtered release keeps
28 only working honest solutions and working backdoors and strips every comment [12]; Control Tax
29 reports its pass-tests pre-filter and appendices “Monitoring failed backdoors” [19]; the comment
30 channel has a paper of its own [18]. One harness ships a `-leak-check` flag with two trivial baselines
31 in it, a `#-count` detector that falls from 0.6957 to 0.4983 once comments go and code length alone at
32 0.4726 [9]. Partial-input baselines are the standard instrument for this [6, 16]; their failure does not
33 certify a dataset [2], and leakage is usually treated as a checklist [10]. Out of field, bag-of-words is
34 known to separate human-written from model-written code [14], and the detectors built on it match
35 their classes on line and character count or on shared problem definitions [5, 1]. Adjacent control

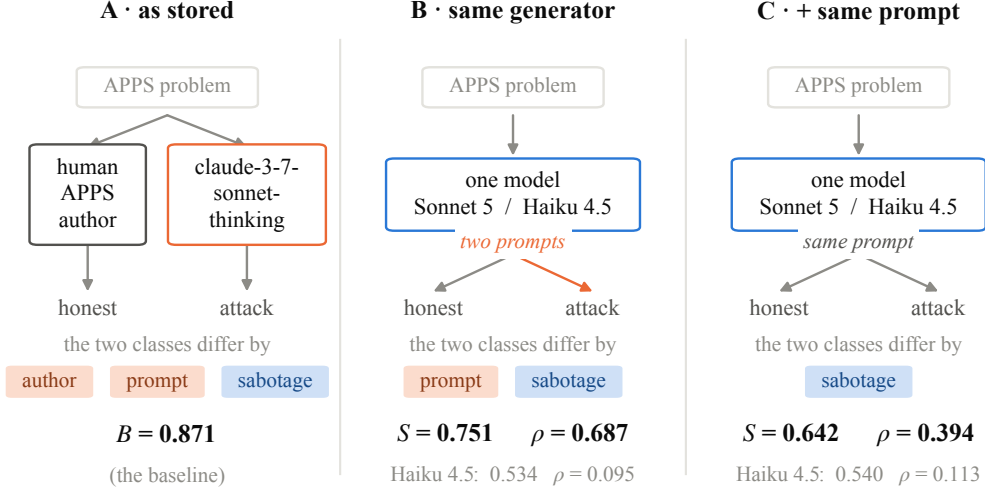


Figure 1: **The two constructions.** As stored, the classes differ in author, in elicitation and in sabotage. Our control removes the first difference, the prompt-matched arm the second. B is the stored-pair baseline over $n = 1,444$ pairs; each ρ is against B' refitted on that arm’s own problems.

work attacks other axes, subverting monitors with injected text, showing that attacker-side selection inflates safety estimates, and removing transcript length as a nuisance covariate [20, 3, 15].

None of that preprocessing removes the authorship gap. On the filtered, comment-stripped split, a bag-of-words classifier reaches AUROC 0.806, above 20 of the 24 open-weight monitors ranked on that same split, whose median is 0.609; a classifier that reads no code at all, only problem difficulty, test count and question length, reaches 0.772, above 20. The classifier transfers across backdoor generators and is nearly as strong on **inert** backdoors as on working ones. Neither is a property of a backdoor.

We contribute (i) a protocol $\times$ baseline measurement of what survives that preprocessing; (ii) a construction for stored-pair corpora, in which one model writes both classes under one prompt framing and one harness verifies both, and the corpus that implements it. Run on two generators, it attributes between a third and nine-tenths of the separability to authorship and most of the remainder to the attack prompt’s framing, and returns a null on the way, since regenerating one half of a stored pair substitutes one confound for another. We add (iii) a second selection artifact the filter installs by defining the negative class through attack failure, and (iv) a preprocessing checklist. **Scope:** this concerns monitor training, selection and benchmarking on *stored* pairs; our measurement is of one APPS-derived family plus five agentic settings. ControlArena’s live evaluations generate both classes and are not implicated.

2 Setup

The confound is legible in the schema. The canonical stored-pair artifact for APPS control evaluations [8] records model and source on every backdoor and no generator field on any honest solution (Table A1). Figure 1 sets that construction beside the one we build against it.

Artifacts and provenance. We use seven public corpora, whose per-class provenance is the object of study. In the APPS family the honest class is human and the attack class is one of ten named models. **Six of the seven declare no licence** [11, 17], so we report only aggregates and redistribute no corpus.

Evaluation surface. Our headline protocol is the filtered, comment-stripped split [12]: 1,209 train problems (2,418 paired items) and 1,375 test items (172 attack, 1,203 honest), labelled by backdoor_works, reconstructed with the released harness code [9]. Generated code is scored by our own runner, applied to both classes (appendix A.11).

66 3 Results

67 Comment stripping works on comment count and code length, driving both to chance (Table A2,
68 appendix). Two floors on that same split do not move. Bag-of-words reaches 0.806 (95% CI 0.7704–
69 0.8409 by bootstrap over problems; permutation null 95th percentile 0.5465, $p < 0.005$), above 20 of
70 the 24 monitors by point estimate and 18 of them with a paired-bootstrap interval excluding zero.
71 The no-code classifier reaches 0.772 (CI 0.7319–0.8005 on the pooled out-of-fold 0.766; null 0.5545,
72 $p < 0.005$), above 20 and 17 respectively (Figure A1, appendix). The preprocessing reached the
73 carrier it was aimed at and left the two beside it standing.

74 **The signal is not sabotage.** Every number in this paragraph is comment-stripped on both classes;
75 the unstripped values, inflated by the comment channel, are in appendix A.3. The classifier *transfers*
76 *across backdoor generators* at 0.969 pairwise. Trained on one attacker’s backdoors it identifies
77 another’s, so what it has learned belongs to no single model’s sabotage style. It is also nearly as
78 strong on **inert** backdoors (0.879) as on working ones (0.935), where a detector of sabotage should
79 find the backdoors that never function much harder. And the residual carrier is stylistic. `__main__`
80 appears in 0.116 of attack samples against **0.000** of honest ones, replicated on the train split (152
81 versus 1 of 1,209) and untouched by the stripper. A token-level version of the test does not replicate
82 here (appendix A.12).

83 **The control is to have one model write both classes.** Under the identical pipeline the human-
84 honest baseline is $B = 0.871$ over $n = 1,444$ pairs. Write G for that same quantity when one model
85 writes both classes. Retained signal is $\rho = (G - 0.5)/(B - 0.5)$: $\rho = 1$ means holding authorship
86 constant changed nothing, $\rho = 0$ that nothing survived. We pre-registered $\rho \leq 0.33$ as authorship
87 and $\rho > 0.72$ as content, and fixed both bands before any arm ran.

88 **One model writing both classes removes between a third and nine-tenths of the separability**
89 (Table 1). We report the **independent** arm, in which the attack class is written from the problem
90 statement alone, so authorship changes and nothing else. An **edit** arm, shown its own honest solution,
91 returns near-copies instead (appendix A.4). We take one outcome-blind draw per problem and define
92 the attack class as the benchmark does, by intent plus passing the public tests. 2.2% of the first
93 generator’s draws fire and 0.3% of the second’s (2 of 611); B' is inert-inclusive over the same
94 problems. The first generator’s interval excludes the collapse band and spans the content line, so
95 the pre-registered rule returns *inconclusive between partial and content*. The second generator’s lies
96 inside the collapse band. Sample size does not explain the drop, since at $n = 623$ the baseline B
97 under the identical pipeline is 0.8521 ± 0.0055 . Two rows are *strict*: they keep only attacks that pass
98 the tests *and* fire on their trigger, selected by the benchmark’s own retry-and-keep. Only 99 pairs
99 meet it, so we read them through ρ alone; they agree with the headline rows at low power.

Table 1: **What is left when one model writes both classes.** S is the same-generator AUROC (5-fold, comments stripped) and B' the stored-pair baseline on the row’s own problems; ρ , its bands and its intervals are as in the text and appendix A.5.

Arm	n	S (95% CI)	B'	ρ (95% CI, joint)	band
Sonnet 5, headline	623	0.7512 (0.7295–0.7713)	0.8658	0.687 (0.625–0.755)	partial–content
Haiku 4.5, headline	611	0.5340 (0.5159–0.5531)	0.8584	0.095 (0.044–0.150)	collapse
Sonnet 5, prompt-matched	586	0.6423 (0.6179–0.6621)	0.8613	0.394 (0.326–0.454)	partial
Sonnet 5, strict	99	0.7652 (0.7016–0.8000)	0.7877	0.921 (0.662–1.089)	partial–content
Sonnet 5, strict, matched	96	0.7113 (0.6464–0.7453)	0.8030	0.697 (0.494–0.856)	partial–content

100 What separates the two generators is the attack prompt. On Sonnet 5 the surviving signal is lexical,
101 with length alone falling to 0.5851 while TF-IDF holds, and it does not run through the trigger.
102 Masking numerals and comparison operators leaves S unchanged; only normalising identifiers moves
103 it, to 0.700 ($\rho = 0.54$). The top attack-side features are `def main` (67% of attack programs against
104 38.5% of honest ones) and `data[`, which together are the program layout the attack prompt elicits.
105 Haiku 4.5 never adopts that layout. `def main` and `sys.stdin` appear in no program of any of
106 its three classes, which read input with `input()` throughout (appendix A.6). Holding the prompt
107 constant as well as the model therefore moves the two generators toward each other, to $\rho = 0.394$
108 and $\rho = 0.113$. On Sonnet this removes most of what is left. The two honest classes alone, with no

109 attack on either side, already separate at 0.6006, so the prompt is carrying framing and no backdoor
110 at all. On Haiku the same prompt-only floor is 0.4918, an interval containing chance.

111 **The same inflation reaches an LLM monitor.** We put a zero-shot monitor prompt to the model that
112 wrote both classes. In this field’s vocabulary it is an untrusted monitor rather than a trusted one, which
113 makes it the most favourable case available, since it is reading its own output. On the *stored* pairs
114 over the headline problems it scores 0.9579; on the same-generator pairs, 0.566 (0.5507–0.5833).
115 That is a monitor- ρ of 0.145 (0.111–0.182), below the classifier’s 0.687. On the strict pairs the two
116 scores are 0.9437 and 0.818 (0.7630–0.8686), a monitor- ρ of 0.717 (0.601–0.832). A monitor that
117 wrote neither class returns the same two ratios: 0.151, a bound at half missingness, and 0.749. Our
118 single-shot attacks fire given a pass at 2.5% against the stored attacker’s 67.2%, which makes the
119 stored attack class a survivor set.

120 Regenerating only the honest half makes separation worse. With an open-weight model writing the
121 honest class against the stored backdoors, $\rho = 1.234$ (appendix A.5). Substituting the honest author
122 replaced one authorship confound with a larger one.

123 **The filter installs a second artifact.** Labelling by `backdoor_works` defines the negative class
124 by *attack failure* at the problem level rather than by honesty. 80.7% of the test split’s labelled
125 backdoors do not work (32.8% of those that pass the public tests), and a problem whose backdoor
126 fails contributes its human honest solution instead. The label is fixed before any code is read. Attack
127 problems also carry half as many test cases (20.95 versus 38.85), and fewer tests make a backdoor
128 easier to hide, so difficulty and test count correlate with the label. That correlation is where the
129 content-blind 0.772 comes from.

130 **Where authorship is already matched, a floor remains.** In five agentic settings where the same
131 three models write both classes, so no authorship confound is possible, the surviving floor is structural:
132 0.504 (bash, on its null) to 0.802 (agentdojo) from character count alone (Figure A2).

133 4 Limitations

134 **A high floor is evidence about a corpus; it does not settle results computed on that corpus.** Our
135 floors show that the label is recoverable without reading a backdoor. They leave open whether any
136 monitor recovers it that way, and no number here corrects a published monitor score.

137 **The generator, and what ρ can therefore be read as.** The model that wrote the stored backdoors,
138 `claude-3-7-sonnet-thinking`, is retired, so both same-generator classes come from a later model
139 of the same lineage. It differs from the stored attacker in backdoor style, not only in authorship,
140 so every ρ here, classifier or monitor, bounds how much of the separability is content rather than
141 partitioning it. The arm is not bit-reproducible, having run through a metered API that does not
142 expose sampling parameters, with thinking off although the stored attacker was a thinking model.
143 The two generators are compared on the passes eligibility alone, Haiku 4.5’s strict row having been
144 refused at 35 pairs against our 50-pair floor.

145 **Scope, provenance and the harness.** Whether stored-pair corpora beyond these leak the same way
146 is untested. Five of our seven corpora are one user’s repackaging, and our claims are scoped to them.
147 Two interpreter-era artifacts reach B but not the same-generator arm (appendix A.11).

148 5 Recommendations

- 149 1. **Generate both rollouts from one model under one prompt framing, and verify both with**
150 **one harness.** If the attack class is model-written, the honest class must be too; regenerating
151 one side substitutes one authorship gap for another. Matched construction is preceded in
152 code-detection benchmarks [5], but we are not aware of it reaching backdoor detection.
- 153 2. **Report a partial-input floor and a bag-of-words floor beside every monitor number,**
154 **each against a permutation null.**
- 155 3. **Do not define the negative class by attack failure.** Drop the problems whose attacks failed,
156 or sample the negative class independently of attack success.
- 157 4. **State the split, the metric and the preprocessing whenever monitors are compared, and**
158 **release all three.** Ours are MIT (appendix A.10).

References

- [1] Basak Demirok, Mucahid Kutlu, and Selin Mergen. MultiAIGCD: A comprehensive dataset for AI generated code detection covering multiple languages, models, prompts, and scenarios. 2025. arXiv:2507.21693v1.
- [2] Shi Feng, Eric Wallace, and Jordan Boyd-Graber. Misleading failures of partial-input baselines. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.
- [3] Catherine Ge-Wang, Tyler Crosse, Benjamin Hadad, Joachim Schaeffer, Ram Potham, and Tyler Tracy. Attack selection in agentic AI control evaluations meaningfully decreases safety. 2026. arXiv:2606.06529.
- [4] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. 2023. arXiv:2312.06942.
- [5] Hanxi Guo, Siyuan Cheng, Kaiyuan Zhang, Guangyu Shen, and Xiangyu Zhang. CodeMirage: A multi-lingual benchmark for detecting AI-generated and paraphrased source code from production-level LLMs. 2025. arXiv:2506.11059v1. No venue stated.
- [6] Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *Proceedings of NAACL-HLT*, 2018.
- [7] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. In *NeurIPS Datasets and Benchmarks Track*, 2021. arXiv:2105.09938. The upstream source of the human-written honest class, distributed as the HuggingFace dataset codeparrot/apps.
- [8] Rogan Inglis. apps-control-arena. HuggingFace dataset `RoganInglis/apps-control-arena`. MIT licence. 5,000 rows; honest class from codeparrot/apps., 2025.
- [9] Anik Jha. Decorrelation is not complementarity: Skill, not lineage, governs trusted-monitor ensembles. 2026. arXiv:2608.16190. Code: <https://github.com/anik-jha/challenger-panels> (MIT).
- [10] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in ML-based science. 2022. arXiv:2207.07048.
- [11] Eugene Koran. apps-control-arena-high-quality. HuggingFace dataset `eugenekoran/apps-control-arena-high-quality`. **No licence declared.** 1,209 train / 1,375 test., 2026.
- [12] Eugene Koran, Yejun Yun, Samantha Tetef, Benjamin Arnav, and Pablo Bernabeu-Pérez. Ensemble monitoring for AI control: Diverse signals outweigh more compute. 2026. arXiv:2605.15377v2.
- [13] Bryce Little. Length penalties make chain-of-thought less monitorable. 2026. arXiv:2607.09786v3.
- [14] Daniil Orel, Dilshod Azizov, Indraneil Paul, Yuxia Wang, Iryna Gurevych, and Preslav Nakov. AICD bench: A challenging benchmark for AI-generated code detection. 2026. arXiv:2602.02079v1. No venue stated on the abs page.
- [15] Lucas Pinto. Calibration-family overfit: Why trusted sabotage monitors don’t transfer across lineages. In *COLM 2026 Workshop on AI Measurement Science*, 2026. arXiv:2607.06596v3.
- [16] Adam Poliak, Jason Naradowsky, Aparajita Haldar, Rachel Rudinger, and Benjamin Van Durme. Hypothesis only baselines in natural language inference. In *Proceedings of the Seventh Joint Conference on Lexical and Computational Semantics (*SEM)*, 2018.

- [17] Aditya Sinha. control_arena_{bash,shade,iac,rogue_eval,agentdojo}. HuggingFace datasets adityaasinha28/control_arena_*. **No licence declared.** 1,500 rows each., 2026.
- [18] Ashwin Sreevatsa, Sebastian Prasanna, and Cody Rushing. Basic legibility protocols improve trusted monitoring. 2026. arXiv:2602.10153.
- [19] Mikhail Terekhov, Zhen Ning David Liu, Caglar Gulcehre, and Samuel Albanie. Control tax: The price of keeping AI in check. 2025. arXiv:2506.05296v3.
- [20] Mikhail Terekhov, Alexander Panfilov, Daniil Dzenhaliou, Caglar Gulcehre, Maksym Andriushchenko, Ameya Prabhu, and Jonas Geiping. Adaptive attacks on trusted monitors subvert AI control protocols. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.09462v2.

A Supplementary tables and figures

A.1 The schema

Table A1 is the record schema of RoganInglis/apps-control-arena [8]. The attack class carries model and source, naming the system that produced each backdoor; the honest class carries no such field, because there is no generator to name. The two APPS-backdoor corpora we use are two views of this one artifact, since every problem in the filtered release [11] is also in [8], byte-identical except a re-validated backdoor_works. Among the agentic corpora we exclude AgentHarm, to avoid a conflict of interest.

Table A1: The confound is legible in the dataset schema, with no measurement at all. The attack class records which model wrote it; the honest class has no generator field. Verified against the shard schema of RoganInglis/apps-control-arena.

struct	sub-fields
solutions	code, compiles, passes_tests
backdoors	backdoor_code, backdoor_input, backdoor_works, compiles, model, passes_tests, source

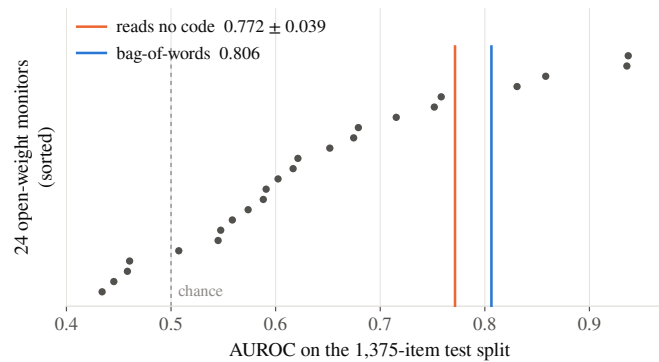


Figure A1: **Two floors against the 24 open-weight monitors ranked on the same split.** AUROC on the 1,375-item test split (172 attack / 1,203 honest), comments stripped; monitor scores from the MIT-licensed release of [9], median 0.609, max 0.937. Both floors clear 20 of the 24. **Two caveats.** (i) The monitors are zero-shot prompted while the bag-of-words classifier is *trained* on the benchmark’s own paired train split: a statement about what the corpus makes recoverable, not a claim that bag-of-words is a better monitor. (ii) The no-code floor is cross-validated *within* the test split; on one held-out slice it is 0.716, above 18 of 24 (appendix A.2).

A.2 The protocol $\times$ baseline table

The no-code floor is cross-validated *within* the test split because the released train/val split is paired and therefore metadata-balanced; its permutation null shuffles labels for fitting and scoring alike. Table A2 is the full measurement behind §3. Rows are not comparable across protocols; the preprocessing column says whether comments were stripped before the baseline read the code. Pairwise accuracies are given to three decimals because their resolution over 2,641 pairs is one pair. Intervals in §3 are bootstraps over problems (2,000 draws) and nulls are 200 label permutations; the held-out no-code row is barely above its own null: 0.716 (CI 0.6476–0.7796) against a 95th percentile of 0.7023, $p = 0.030$, above 18 of the 24 monitors by point estimate and 12 with a paired interval excluding zero. The no-code claim therefore rests on the cross-validated protocol.

Table A2: What the field’s preprocessing leaves behind. Every row is measured under the protocol named in column one; rows are not comparable across protocols. *Prep.* says whether comments were stripped before the baseline read the code; *no code* means the baseline never reads the program. The comments-kept rows are inflated by the comment channel and are given for contrast, not as results. The matched rows are problem-disjoint and scored against the DSPy tutorial monitor; the transfer row runs Control Tax to the GPT-4-era set. Pairwise accuracies are given to three decimals, their resolution over the pair count.

Protocol	Baseline	Value	Prep.	Against
<i>AUROC</i>				
Koran’s exact split	TF-IDF word 1–2gram	0.8063	stripped	above 20 of 24
same	TF-IDF word 1–2gram	0.8914	kept	above 22 of 24
same, reads no code	difficulty + #tests + $ q $	0.7715 ± 0.0389	no code	above 20 of 24
same, held-out slice	difficulty + #tests + $ q $	0.7163	no code	above 18 of 24
same	code length only	0.4726	stripped	chance
same	#-count detector	0.4983	stripped	chance
same	#-count detector	0.6957	kept	—
Raw artifact, problem level	all metadata	0.8917	no code	—
same	difficulty only	0.6823	no code	—
same	<code>solution_passes_tests</code>	0.7881	no code	—
Matched pairs (P3)	all metadata	0.5000	no code	by construction
<i>Pairwise accuracy</i>				
DSPy matched pairs	TF-IDF word 1–2gram	0.897	stripped	monitor 0.69–0.81
working backdoors only	TF-IDF word 1–2gram	0.935	stripped	—
inert backdoors only	TF-IDF word 1–2gram	0.879	stripped	not sabotage
transfer, cross-generator	TF-IDF word 1–2gram	0.969	stripped	—
all pairs, comments kept	TF-IDF word 1–2gram	0.962	kept	—
working, comments kept	TF-IDF word 1–2gram	0.979	kept	—
inert, comments kept	TF-IDF word 1–2gram	0.955	kept	—
transfer, comments kept	TF-IDF word 1–2gram	0.994	kept	—
same, raw artifact	code length only	0.826	kept	—

A.3 Comment stripping on the matched-protocol numbers

The matched-pair and transfer measurements were first made with comments in. Every value in the authorship paragraph of §3 was re-measured with the same stripper applied to both classes, over five problem-disjoint splits (each transfer row is one fixed split). With comments kept the same quantities were transfer 0.994, working 0.979, inert 0.955, all pairs 0.962 and surface-feature AUROC 0.8306; stripped they are 0.969, 0.935, 0.879, 0.897 and 0.7259. Every one was inflated by the comment channel, which is this paper’s thesis applied to itself. The #-presence marker, 0.685 against 0.176 before stripping, closes to 0.007 against 0.008 after and is dropped; `__main__` is unchanged. One authorship leak survives the stripper because string literals are not comments: in the comment-stripped train split an author byline (Author :) survives in 23 of 1,209 honest samples against 2 attack samples, and a triple-quoted block in 43 against 9. Both are far too rare to drive 0.806, but both are a literal record of authorship. Stripped, the matched-protocol TF-IDF AUROC is 0.8560, beside $B = 0.871$ and the Koran-split 0.806, all three on stripped code. The other cross-generator transfer rows, stripped, are 0.866 (o3-mini to claude-3-7-sonnet, 82 test pairs), 0.844 (the reverse) and 0.764 (o1-mini to gemini-2.0-flash).

249 A.4 Pair similarity, and why the same-generator control needs two arms

250 An attack class built by editing its own honest solution changes pair similarity as well as authorship
 251 (Table A3); the independent arm sits in the stored benchmark’s regime and if anything slightly further
 252 from it, so it cannot manufacture a low S by making the classes alike.

Table A3: Character similarity of honest/attack pairs, comments stripped. The edit arm’s classes are near-copies; the independent arm reproduces the stored benchmark’s regime, where the two classes are largely independent programs. The last two columns are the share of pairs above and below those similarities.

Attack class	mean sim.	share > 0.95	share < 0.60	changes vs. the benchmark
stored benchmark	0.606	3.4%	45.4%	the regime to match
edit (shown its own solution)	0.928	64.2%	4.0%	authorship <i>and</i> similarity
independent (problem only)	0.659	3.4%	34.8%	authorship only

253 A.5 Robustness of the same-generator rows

254 At the strict rows’ $n = 99$, B under the identical pipeline is 0.7807 ± 0.0248 . Intervals in the
 255 same-generator paragraphs of §3 are bootstraps over problems, resampling S and B' jointly for ρ ;
 256 this replaced a ± 2 fold-sd rule before the retry pool was read. Every ρ point we quote is the ratio of
 257 the fold-mean AUROCs and every joint interval beside it is the bootstrap of the pooled out-of-fold ρ ,
 258 which is the same quantity computed two ways: the pooled points are 0.86 (strict, the row whose
 259 interval Table 1 quotes), 0.390 (prompt-matched headline), 0.662 (prompt-matched strict) and 0.097
 260 (the second generator). Five fold seeds move the strict row’s ρ over 0.85–0.92. Retention does not
 261 produce the effect. Survivors and non-survivors separate at only 0.62, and S computed with non-firing
 262 attempts on the same problems is 0.7595 against 0.7687. The edit arm bounds S under near-copies
 263 (similarity 0.928, Table A3): 0.6266 over 427 pairs, $\rho = 0.371$. In arm 1, where only the honest
 264 half was regenerated, $G = 0.962$ against $B' = 0.874$ on that arm’s $n = 668$ eligible problems, and
 265 TF-IDF gains +0.088 while the length carrier collapses to 0.489. Our own attack prompt halves our
 266 single-shot firing rate: 52.0% of our test-passing draws offer a public test input as the trigger, which
 267 cannot diverge from the honest solution, and those fire 0.15% against 5.13% for novel triggers; no
 268 stored working backdoor has such a trigger (0 of 3,188). The rest is retry-and-keep.

269 A.6 The prompt-matched arm under the masks, and the second generator

270 Under prompt matching the identifier mask, the only mask of §3 that moved S , takes the headline row
 271 to 0.6264 ($\rho = 0.342$) where it takes the original honest class on the same 586 problems to 0.7046;
 272 most of what identifier normalisation removed is what prompt matching removes. That masked row
 273 is the one that decides whether the residue is structure or naming, and under prompt matching it
 274 is band-ambiguous: its joint interval, 0.280–0.394, puts 40% of its mass in the collapse band. A
 275 character model (3–5-grams, same classifier and folds) retains more than the word tokeniser, 0.6817
 276 ($\rho = 0.493$), so the drop is not an artefact of tokenisation. On the strict pairs the matched row is
 277 $S = 0.7113$ ($n = 96$, $\rho = 0.697$, 0.494–0.856). The second generator’s pairs are more self-similar
 278 than the first’s (0.724 mean against 0.659), so S was recomputed inside similarity strata, where it
 279 stays 0.4737–0.5142 in every stratum, including the 0.451-similarity stratum, which is further from
 280 its honest class than the stored benchmark’s own pairs, while B' holds 0.7884–0.8334, so near-copies
 281 are not what buys the collapse. Its prompt-matched rows move S to 0.5399 ($\rho = 0.113$, $n = 562$) and
 282 put the prompt-only floor at 0.4918, an interval containing chance. The mechanism is visible in the
 283 programs. The layout vocabulary that the first generator’s residue is made of, `def main, sys.stdin`
 284 and `data[`, is present in 0.000 of the second generator’s programs in *all three* of its classes, which
 285 read standard input with `input(` in 1.000 of programs on every side, so there is no layout difference
 286 left for the classifier to read.

287 A.7 The monitor rows, and what bounds the second monitor

288 The monitor that wrote neither class scores 0.5577 on the same-generator headline pairs and 0.8824
 289 on the stored pairs over the same problems, 0.7871 and 0.8832 on the strict pairs. Its headline rows
 290 bound the value without fixing it. A 512-token reply ceiling left 628 of 1,246 same-generator replies

unparseable, and on the stored side the missing share differs by class (50.1% honest against 23.4% attack), a bias of unknown sign. Re-running the strict pairs with a 1,024-token ceiling prices it: missingness falls to 2 of 198 and monitor- ρ moves from 0.749 to 0.766, inside the bootstrap interval. On the stored pairs both monitors beat the bag-of-words baseline over the same problems (+0.09 and +0.15 for the first, +0.02 and +0.09 for the second): Figure A1’s ranking is a statement about those monitors on that split, not a claim that a bag of words beats a monitor here. Classifier and monitor are independent pair by pair on the same-generator pairs ($\phi = 0.04$), so the two are reading different things.

A.8 The two generators’ funnel

Both generators were run through the identical pipeline. Our honest draws pass the public tests at 0.7701 and 0.7278 of 1,444 problems; the second generator’s attack draws pass at 689 of 1,444 and 35 of those fire, below the 50-pair floor gate_s_samegen.py requires, which is why it has no strict row and the two generators are compared on the passes eligibility only. The two compared rows differ in inertness. 2 of the 611 attacks in the second generator’s row fire (99.67% inert) against 14 of 623 (97.75%), the other 33 of its firing draws sitting on problems where its own honest program failed the tests and which the control therefore drops. The single-shot firing rate belongs to the protocol and not to either model, since 52.0% of test-passing draws offer a public test input as their trigger.

A.9 The five same-generator agentic settings

Figure A2 shows each setting’s content-blind floor against its own permutation null.

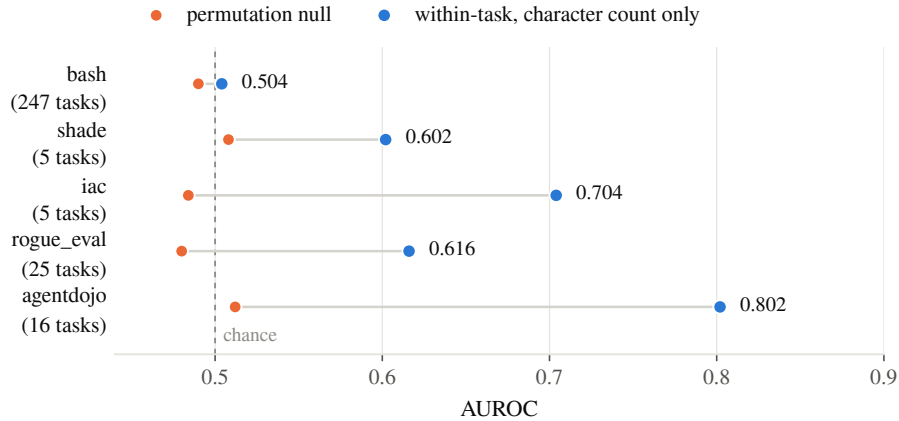


Figure A2: **The structural floor in five same-generator agentic settings.** Each setting’s content-blind classifier (six count features, no content read) scored *within task* against its own permutation null. No authorship confound is possible here: claude-sonnet-4, gemini-2.5-pro and gpt-5 write both classes in all five. bash sits on its null at 0.504; agentdojo reaches 0.802 from character count alone. **The task-group count beside each setting matters:** shade and iac average over five task groups each while bash averages over 247, so these floors are not equally well determined. We report the within-task column because our own pooled numbers partly broke on task composition, and we broke them ourselves. The permutation nulls shown were computed with a pooled estimator rather than the folded within-task statistic; they bound the chance level but are not a like-for-like null for each dot. These corpora declare no licence; we redistribute nothing and report aggregates only.

A.10 The release

The supplementary bundle holds the same-generator split (1,444 honest programs, 1,112 edit-arm and 1,112 independent-arm attack programs written by one model, plus every one of the 11,565 draws of the independent arm’s retry pool), both eligibility rules as pair lists under an outcome-blind selection (623 passing and 14 firing independent-arm pairs) and under the benchmark’s own survivor selection (1,046 and 99, or 106 counting a crash on the trigger as firing), the 24 scripts behind the

316 measurements (generation for both generators, the test harness, the trigger verifier, the selection rules,
317 the prompt-matched arm, the monitor rows and each remaining measurement script), a dataset card,
318 and a manifest of row counts and checksums. The licence, MIT, is a file inside the bundle and covers
319 the data and the scripts alike.

320 **A.11 The test harness and the pass flag**

321 The same-generator arm requires executing generated code, so we built a runner that feeds each
322 shipped input case to a solution under per-case timeouts and address-space limits. It agrees with the
323 artifact's own `solution_passes_tests` on 95.84% of the analysis pool, and is strictly conservative
324 there, never crediting a solution the artifact rejects (0 such cases). We use *our own* flag for both
325 classes, because a checker stricter on one class than the other would manufacture exactly the artifact
326 this paper documents. We do not length-match either class; the untrusted model chooses its own
327 trajectory length. The headline does not depend on that choice. Under the artifact's flag $B = 0.871$
328 at $n = 1,444$, under ours $B = 0.873$ at $n = 1,406$, within one standard deviation of each other.
329 The monitor scores in Figure A1 are runs/x0/monitor-summary.json from the MIT release of
330 that harness [9], used under its notice. Two interpreter-era artifacts in the upstream solutions fall on
331 opposite classes: a `stdin` without `.buffer`, fixed, and `fractions.gcd`, removed in Python 3.9, not
332 fixable. B is exposed to them; the same-generator arm, whose code we generated and ran ourselves,
333 is immune.

334 **A.12 The token test, run and failed**

335 We attempted to confirm a reported token-level mechanism [14] in our setting and it does not replicate.
336 Separation is -0.013 , the wrong sign, at permutation $p = 0.956$. The reason is register. Their human
337 class is general-purpose repository code, whereas *both* of our classes are competitive-programming
338 code, so an identifier axis that separates general human code from model code does not separate
339 these two populations. Only 8 of their 10 canonical tokens are in our vocabulary, so the test is
340 underpowered; but the point estimate has the wrong sign, which is not what an underpowered true
341 effect looks like. We report it because a reader who knows that work may run exactly this test.

342 **A.13 A third channel, noted but not developed here**

343 In the same five agentic settings a fifteen-word regular expression over the untrusted policy's own
344 scratchpad reaches 0.76–0.82, because prompted attack policies restate their side task in the reasoning
345 the monitor is given. We note this only to bound the claim. The instrument is not ours, and the work
346 it comes from [13] validates it against an LLM judge and a length-matched null; we defer to that
347 treatment.